LibraFlow

Guide de Tests Backend — Phases 1 à 4

22 tests • 7 scénarios • php artisan tinker

Légende des couleurs dans les blocs de code

// Commentaire marron italique → ce que le test permet de vérifier
Commande bleue → la commande à copier-coller dans Tinker
=> Résultat vert gras → la réponse attendue
Commande violette → commande système (CMD, pas Tinker)

#	Scénario	Nb tests	Concept validé
1	Authentification & Rôles	6	User, Policies, isAdmin/Biblio/Lecteur
2	Catalogue livres (CRUD + Scopes)	7	Book::search(), available(), relations
3	Emprunt + Observer + Carbon	5	LoanObserver::created(), due_at Carbon
4	Retard & Pénalité	5	Loan::overdue(), calculatePenalty()
5	Retour de livre	5	LoanObserver::updated(), isDirty()
6	File d'attente Réservations	5	Reservation::enAttente(), ordre file
7	Paramètres Admin (Settings)	5	Setting::getValue/setValue()

⚠ **Lance php artisan serve dans un autre CMD avant de commencer**

Scénario 1 — Authentification & Rôles

Objectif

Vérifier que les 3 comptes sont bien créés par UserSeeder avec les bons rôles,
que les méthodes helper isAdmin/isBibliothecaire/isLecteur() fonctionnent,
et que les Policies autorisent et refusent les bonnes actions par rôle.

▶ Ouvre CMD dans le dossier libraflow et lance Tinker

Garde cette fenêtre ouverte pendant TOUT le scénario 1.
Copie-colle chaque commande une par une.

```
cd C:\xampp\htdocs\libraflow

php artisan tinker
```

TEST 1.1 Les 3 comptes existent avec les bons rôles

```
// Vérifie que UserSeeder a créé les 3 utilisateurs avec les bons rôles
User::all()->pluck('role', 'email');

=> Illuminate\Support\Collection {
=>   "admin@libraflow.com"   => "admin",
=>   "biblio@libraflow.com"  => "bibliothecaire",
=>   "lecteur@libraflow.com" => "lecteur",
=> }
```

TEST 1.2 Charger les 3 users dans des variables (à faire UNE FOIS)

```
// Ces variables seront réutilisées dans tous les tests du scénario 1
$admin   = User::where('email','admin@libraflow.com')->first();
$biblio  = User::where('email','biblio@libraflow.com')->first();
$lecteur = User::where('email','lecteur@libraflow.com')->first();

=> = App\Models\User { id: 1, name: "Super Admin", role: "admin" }
=> = App\Models\User { id: 2, name: "Marie Biblio", role: "bibliothecaire" }
=> = App\Models\User { id: 3, name: "Jean Lecteur", role: "lecteur" }
```

TEST 1.3 Méthodes helper — true pour le bon rôle

```
// Chaque méthode doit retourner true uniquement pour le rôle correspondant
$admin->isAdmin();
=> true ✓
$biblio->isBibliothecaire();
=> true ✓
$lecteur->isLecteur();
=> true ✓
```

TEST 1.4 Méthodes helper — false pour les mauvais rôles (vérifications croisées)

```
// Un lecteur ne doit jamais être reconnu comme admin ou bibliothécaire
$lecteur->isAdmin();
=> false ✓
$lecteur->isBibliothecaire();
=> false ✓
$biblio->isLecteur();
=> false ✓
$admin->isLecteur();
=> false ✓
```

TEST 1.5 BookPolicy::create() — qui peut créer un livre ?

```
// BookPolicy::create() : admin et biblio autorisés, lecteur refusé
$book = App\Models\Book::first();

app(App\Policies\BookPolicy::class)->create($admin);
=> true  ← admin autorisé ✓
app(App\Policies\BookPolicy::class)->create($biblio);
=> true  ← bibliothécaire autorisé ✓
app(App\Policies\BookPolicy::class)->create($lecteur);
=> false ← lecteur refusé ✓
```

TEST 1.6 BookPolicy::delete() — seul l'admin peut supprimer

```
// BookPolicy::delete() : seul l'admin est autorisé à supprimer un livre
app(App\Policies\BookPolicy::class)->delete($admin, $book);
=> true  ← admin autorisé ✓
app(App\Policies\BookPolicy::class)->delete($biblio, $book);
=> false ← bibliothécaire refusé ✓
app(App\Policies\BookPolicy::class)->delete($lecteur, $book);
=> false ← lecteur refusé ✓
```

TEST 1.7 LoanPolicy::view() — le lecteur voit seulement SES emprunts

```
// LoanPolicy::view() : le lecteur ne peut voir que ses propres emprunts
$monEmprunt    = App\Models\Loan::where('user_id', $lecteur->id)->first();
$autreEmprunt  = App\Models\Loan::where('user_id', '!=', $lecteur->id)->first();

app(App\Policies\LoanPolicy::class)->view($lecteur, $monEmprunt);
=> true  ← son propre emprunt ✓
app(App\Policies\LoanPolicy::class)->view($lecteur, $autreEmprunt);
=> false ← emprunt d'un autre → refusé ✓
```

TEST 1.8 LoanPolicy::create() — seuls admin et biblio peuvent créer un emprunt

```
// Seul le bibliothécaire ou l'admin enregistre un emprunt au guichet
app(App\Policies\LoanPolicy::class)->create($biblio);
```

```
=> true  ← bibliothécaire autorisé ✓
app(App\Policies\LoanPolicy::class)->create($lecteur);
=> false ← lecteur ne peut pas créer d'emprunt lui-même ✓
```

> ☑ **Fin scénario 1 — NE PAS quitter Tinker, passe directement au scénario 2**
> Les variables $admin, $biblio, $lecteur, $book sont toujours disponibles.

Scénario 2 — Catalogue livres (CRUD + Scopes)

> **Objectif**
> Vérifier que BookSeeder a inséré les 8 livres, que les scopes search() et available() filtrent correctement, et que les relations hasMany book→loans et book→reservations fonctionnent dans les deux sens.

TEST 2.1 BookSeeder — 8 livres en base

```
// Vérifie que BookSeeder a bien créé les 8 livres de démonstration
App\Models\Book::count();
=> 8 ✓
```

TEST 2.2 Scope search() — recherche par TITRE

```
// scopeSearch() utilise LIKE %...% sur la colonne title
App\Models\Book::search('Laravel')->get()->pluck('title');
=> ["Laravel : le guide complet"] ✓
```

TEST 2.3 Scope search() — recherche par AUTEUR

```
// scopeSearch() cherche aussi dans la colonne author (orWhere)
App\Models\Book::search('Coelho')->get()->pluck('title');
=> ["L'Alchimiste"] ✓
```

TEST 2.4 Scope search() — recherche par CATÉGORIE

```
// scopeSearch() cherche aussi dans la colonne category (orWhere)
App\Models\Book::search('Informatique')->get()->pluck('title');
=> ["Introduction aux algorithmes", "Clean Code", "Laravel : le guide complet"] ✓
```

TEST 2.5 Scope search() — terme introuvable retourne collection vide

```
// Un terme inexistant ne doit pas faire planter l'appli mais retourner []
App\Models\Book::search('xyzinexistant')->get()->pluck('title');
```

```
=> [] ← collection vide, pas d'erreur ✓
```

TEST 2.6 Scope available() — livres avec copies disponibles

```
// scopeAvailable() = WHERE available_copies > 0
// Les livres entièrement empruntés (dispo=0) ne doivent PAS apparaître
App\Models\Book::available()->count();
=> Nombre > 0 (livres non entièrement empruntés) ✓

App\Models\Book::where('available_copies', 0)->count();
=> Nombre >= 1 (livres épuisés, ex: Clean Code, Sapiens) ✓
```

TEST 2.7 Filtre par catégorie directement

```
// Filtre SQL simple utilisé dans BookController::index()
App\Models\Book::where('category','Informatique')->count();
=> 3 ✓
App\Models\Book::where('category','Roman')->count();
=> 2 ✓
```

TEST 2.8 Relation hasMany — book → loans

```
// Un Book doit lister ses emprunts via la relation hasMany(Loan::class)
$b = App\Models\Book::where('title','Le Petit Prince')->first();
$b->loans()->count();
=> 1 (emprunt créé par LoanSeeder) ✓
```

TEST 2.9 Relation hasMany — book → reservations

```
// Un Book doit aussi lister ses réservations via hasMany(Reservation::class)
$sapiens = App\Models\Book::where('title','Sapiens')->first();
$sapiens->reservations()->count();
=> 1 (réservation créée par LoanSeeder) ✓
```

TEST 2.10 Vérification des copies d'un livre épuisé

```
// Clean Code a 1 copie totale et 0 dispo car entièrement emprunté
$cc = App\Models\Book::where('title','Clean Code')->first();
echo "Total: {$cc->total_copies} | Dispo: {$cc->available_copies}";
=> Total: 1 | Dispo: 0 ✓
```

> ✅ **Fin scénario 2 — NE PAS quitter Tinker**
> Continue avec le scénario 3 directement.

Scénario 3 — Emprunt + Observer + Carbon

Objectif

Vérifier que la création d'un Loan déclenche LoanObserver::created() qui décrémente available_copies automatiquement sans code manuel. Vérifier que due_at est calculé avec Carbon::now()->addDays($duree) en lisant la durée depuis la table settings.

TEST 3.1 Lire la durée d'emprunt depuis Settings

```
// La durée n'est pas codée en dur — elle vient de la table settings
// Setting::getValue() retourne toujours une string → cast en int
$duree = (int) App\Models\Setting::getValue('duree_emprunt', 14);
echo "Durée configurée : {$duree} jours";
=> Durée configurée : 14 jours ✓
```

TEST 3.2 État AVANT l'emprunt — noter available_copies

```
// Note la valeur avant pour pouvoir vérifier le décrément après
$livre = App\Models\Book::where('title','Le Petit Prince')->first();
echo "Avant emprunt — Dispo: {$livre->available_copies}";
=> Avant emprunt — Dispo: 2 ✓
```

TEST 3.3 Créer l'emprunt avec Carbon — vérifier borrowed_at et due_at

```
// Crée l'emprunt. L'Observer se déclenche automatiquement après le create()
// due_at = Carbon::now()->addDays(14) → date exacte dans 14 jours
$emprunt = App\Models\Loan::create([
    'book_id'     => $livre->id,
    'user_id'     => $lecteur->id,
    'borrowed_at' => Carbon\Carbon::now(),
    'due_at'      => Carbon\Carbon::now()->addDays($duree),
]);

echo 'Emprunté le : ' . $emprunt->borrowed_at->format('d/m/Y');
=> Emprunté le : [date d'aujourd'hui] ✓
echo 'À rendre le : ' . $emprunt->due_at->format('d/m/Y');
=> À rendre le : [aujourd'hui + 14 jours] ✓
```

TEST 3.4 Observer::created() — available_copies décrémenté AUTOMATIQUEMENT

```
// POINT CLÉ : LoanObserver::created() a fait book->decrement() tout seul
// fresh() recharge l'objet depuis la base pour avoir la vraie valeur
$livre->fresh()->available_copies;
=> 1  ← était 2 avant l'emprunt, maintenant 1 ✓
```

TEST 3.5 Vérifier qu'on ne peut pas emprunter un livre épuisé

```
// La logique du controller vérifie available_copies > 0 avant de créer
$livreEpuise = App\Models\Book::where('available_copies', 0)->first();
if ($livreEpuise->available_copies <= 0) {
    echo 'BLOQUÉ : plus d\'exemplaires disponibles ✓';
} else {
    echo 'Disponible';
}
=> BLOQUÉ : plus d'exemplaires disponibles ✓
```

TEST 3.6 Relations inverses — emprunt connaît son livre et son lecteur

```
// Les relations belongsTo (Loan→Book et Loan→User) doivent fonctionner
$emprunt->book->title;
=> "Le Petit Prince" ✓
$emprunt->user->name;
=> "Jean Lecteur" ✓
```

> ✅ **Fin scénario 3 — NE PAS quitter Tinker**
> La variable $emprunt sera utilisée dans le scénario 5 (retour).

🔴 Scénario 4 — Retard & Pénalité (overdue + calculatePenalty)

> **Objectif**
> Vérifier que Loan::overdue() détecte les emprunts avec due_at < now() ET returned_at IS NULL.
> Vérifier que calculatePenalty() = jours_de_retard × penalite_journaliere (depuis settings).
> Vérifier les cas limites : emprunt rendu, emprunt dans les délais → pénalité = 0.

TEST 4.1 Créer un emprunt artificiellement en retard de 6 jours

```
// On recule due_at dans le passé (6 jours) pour simuler un retard réel
$livreR = App\Models\Book::where('title',"L'Alchimiste")->first();
$empruntRetard = App\Models\Loan::create([
    'book_id'     => $livreR->id,
    'user_id'     => $lecteur->id,
    'borrowed_at' => Carbon\Carbon::now()->subDays(20),
    'due_at'      => Carbon\Carbon::now()->subDays(6),
]);
=> Loan { id: X, due_at: [il y a 6 jours], returned_at: null } ✓
```

TEST 4.2 Scope overdue() — détecte tous les emprunts en retard

```
// scopeOverdue() = WHERE due_at < NOW() AND returned_at IS NULL
```

```
// Doit inclure l'emprunt du LoanSeeder + celui créé ci-dessus
App\Models\Loan::overdue()->count();
=> 2 ou plus ✓

// Liste les titres des livres en retard
App\Models\Loan::overdue()->with('book')->get()->pluck('book.title');
=> ["Clean Code", "L'Alchimiste"] ✓
```

TEST 4.3 calculatePenalty() — 6 jours × 100 FCFA = 600 FCFA

```
// calculatePenalty() lit penalite_journaliere depuis settings puis multiplie
echo 'Taux : ' . App\Models\Setting::getValue('penalite_journaliere') . ' FCFA/j';
=> Taux : 100 FCFA/j ✓

$empruntRetard->calculatePenalty();
=> 600.0  ← 6 jours × 100 FCFA ✓
```

TEST 4.4 calculatePenalty() = 0 pour un emprunt dans les délais

```
// Un emprunt non échu ne doit JAMAIS générer de pénalité (due_at > now)
$empruntOk = App\Models\Loan::where('due_at','>',Carbon\Carbon::now())
                            ->whereNull('returned_at')->first();
$empruntOk->calculatePenalty();
=> 0.0  ← aucune pénalité car pas encore en retard ✓
```

TEST 4.5 calculatePenalty() = 0 pour un emprunt déjà rendu

```
// Si returned_at est rempli, la pénalité est figée — ne se recalcule pas
$rendu = App\Models\Loan::whereNotNull('returned_at')->first();
$rendu->calculatePenalty();
=> 0.0  ← livre rendu, pénalité déjà enregistrée dans penalty_amount ✓
```

TEST 4.6 Scope active() — emprunts EN COURS (returned_at IS NULL)

```
// scopeActive() = whereNull(returned_at) : tous les emprunts non rendus
App\Models\Loan::active()->count();
=> Nombre > 0 (emprunts en cours incluant les retards) ✓
```

> ✅ **Fin scénario 4 — NE PAS quitter Tinker**
> La variable $empruntRetard sera utilisée dans le scénario 5.

🟠 Scénario 5 — Retour de livre (Observer inverse + Mail log)

Objectif

Vérifier que update(returned_at) déclenche LoanObserver::updated() qui incrémente available_copies via isDirty('returned_at'). Vérifier que penalty_amount est persisté. Vérifier la trace du Mailable dans storage/logs/laravel.log.

TEST 5.1 État AVANT le retour — noter available_copies

```
// Note la valeur AVANT le retour pour comparer après
$avant = $empruntRetard->book->fresh()->available_copies;
echo "Avant retour — Dispo : {$avant}";
=> Avant retour — Dispo : 0 (ou valeur actuelle) ✓
```

TEST 5.2 Enregistrer le retour avec pénalité calculée

```
// update() avec returned_at → déclenche LoanObserver::updated()
// isDirty('returned_at') détecte le changement → book->increment()
$penaliteFinale = $empruntRetard->calculatePenalty();
$empruntRetard->update([
    'returned_at'    => Carbon\Carbon::now(),
    'penalty_amount' => $penaliteFinale,
]);

echo 'Pénalité : ' . $empruntRetard->fresh()->penalty_amount . ' FCFA';
=> Pénalité : 600 FCFA ✓
```

TEST 5.3 Observer::updated() — available_copies incrémenté AUTOMATIQUEMENT

```
// POINT CLÉ : LoanObserver::updated() incrémente book available_copies seul
// fresh() est indispensable pour relire depuis la base (pas depuis le cache)
$apres = $empruntRetard->book->fresh()->available_copies;
echo "Après retour — Dispo : {$apres}";
=> Après retour — Dispo : 1  ← a bien augmenté de 1 ✓
```

TEST 5.4 Vérifier returned_at et penalty_amount persistés en base

```
// Les deux champs doivent être correctement sauvegardés en base de données
$check = App\Models\Loan::find($empruntRetard->id);
echo $check->returned_at->format('d/m/Y H:i');
=> [date et heure d'aujourd'hui] ← returned_at rempli ✓
echo $check->penalty_amount;
=> 600  ← penalty_amount enregistré ✓
```

TEST 5.5 Vérifier la trace du Mailable dans les logs

```
// Quitte Tinker pour lancer une commande système dans CMD
exit
```

⚠ **Cette commande se lance dans CMD (pas dans Tinker)**

Cherche la trace de l'email envoyé au lecteur lors du retour.
Fonctionne uniquement si MAIL_MAILER=log dans .env

```
findstr "Votre livre est disponible" storage\logs\laravel.log
```

```
=> [Si réservation en attente] → ligne avec le sujet du mail ✓
=> [Si aucune réservation]     → rien affiché (normal) ✓
```

▶ **Relance Tinker pour le scénario 6**

Tape la commande ci-dessous puis recharge les variables.

```
php artisan tinker
```

```
// Recharge les variables utilisateurs après relance de Tinker
$admin   = App\Models\User::where('email','admin@libraflow.com')->first();
$biblio  = App\Models\User::where('email','biblio@libraflow.com')->first();
$lecteur = App\Models\User::where('email','lecteur@libraflow.com')->first();
=> Variables rechargées ✓
```

Scénario 6 — File d'attente Réservations

Objectif

Vérifier que le scope enAttente() filtre correctement, que la file est ordonnée par reserved_at (FIFO), qu'on ne peut pas réserver deux fois le même livre, et que le 1er en file passe à "notifie" lors d'un retour.

TEST 6.1 Scope enAttente() — réservations du seeder

```
// scopeEnAttente() = WHERE status = "en_attente"
App\Models\Reservation::enAttente()->count();
=> 1 (créée par LoanSeeder) ✓
```

TEST 6.2 Trouver un livre épuisé pour la réservation

```
// La réservation n'est autorisée que si available_copies = 0
$livreFull = App\Models\Book::where('available_copies', 0)->first();
echo "Livre : {$livreFull->title} | Dispo : {$livreFull->available_copies}";
=> Livre : Sapiens | Dispo : 0 ✓
```

TEST 6.3 Créer une 2e réservation — vérifier l'ordre de la file

```
// La file est ordonnée par reserved_at : premier arrivé = premier servi
```

```
App\Models\Reservation::create([
    'book_id'     => $livreFull->id,
    'user_id'     => $biblio->id,
    'reserved_at' => Carbon\Carbon::now(),
    'status'      => 'en_attente',
]);

// Affiche la file dans l'ordre chronologique
App\Models\Reservation::where('book_id', $livreFull->id)
    ->enAttente()->orderBy('reserved_at')->get()
    ->each(fn($r) => print("{$r->user->name} — {$r->reserved_at->format('d/m
H:i')}\n"));
=> Jean Lecteur — [date seeder]   ← 1er en file ✓
=> Marie Biblio — [maintenant]    ← 2e en file ✓
```

TEST 6.4 Doublon bloqué — impossible de réserver deux fois

```
// ReservationController::store() vérifie l'existence avant de créer
$doublon = App\Models\Reservation::where('book_id', $livreFull->id)
    ->where('user_id', $biblio->id)
    ->where('status', 'en_attente')->exists();
echo $doublon ? "Déjà réservé → BLOQUÉ ✓" : "Pas de doublon";
=> Déjà réservé → BLOQUÉ ✓
```

TEST 6.5 Notifier le 1er en file — statut → "notifie"

```
// Simule ce que LoanController::retour() fait automatiquement après un retour
// Seul le 1er en file change de statut, le 2e reste "en_attente"
$premier = App\Models\Reservation::where('book_id', $livreFull->id)
    ->enAttente()->orderBy('reserved_at')->first();
echo "1er en file : {$premier->user->name} | Statut : {$premier->status}";
=> 1er en file : Jean Lecteur | Statut : en_attente

$premier->update(['status' => 'notifie']);

// Vérifie que SEUL le 1er a changé, le 2e reste en_attente
App\Models\Reservation::where('book_id', $livreFull->id)
    ->get()->each(fn($r) => print("{$r->user->name} → {$r->status}\n"));
=> Jean Lecteur → notifie    ← notifié ✓
=> Marie Biblio → en_attente ← toujours en attente ✓
```

TEST 6.6 Annulation de réservation — statut → "annulee"

```
// Un lecteur peut annuler SA réservation via ReservationController::destroy()
$maResa = App\Models\Reservation::where('user_id', $biblio->id)
    ->where('status','en_attente')->first();
if($maResa) $maResa->update(['status' => 'annulee']);

App\Models\Reservation::where('user_id', $biblio->id)->first()->status;
```

```
=> "annulee" ✓
```

> ✅ **Fin scénario 6 — NE PAS quitter Tinker**
> Continue avec le scénario 7 directement.

⚙ Scénario 7 — Paramètres Admin (Settings)

> **Objectif**
> Vérifier Setting::getValue() et setValue() (updateOrCreate en base).
> Vérifier que modifier duree_emprunt change immédiatement le due_at des nouveaux emprunts.
> Vérifier que modifier penalite_journaliere change immédiatement calculatePenalty().
> Remettre les valeurs initiales à la fin pour garder la base propre.

TEST 7.1 Lire les 2 paramètres par défaut

```
// Les 2 paramètres ont été insérés par SettingSeeder lors du migrate:fresh --seed
App\Models\Setting::getValue('duree_emprunt');
=> "14" ✓
App\Models\Setting::getValue('penalite_journaliere');
=> "100" ✓
```

TEST 7.2 Compter les settings en base

```
// Il doit y avoir exactement 2 entrées dans la table settings
App\Models\Setting::count();
=> 2 ✓
App\Models\Setting::all()->pluck('value','key');
=> {"duree_emprunt": "14", "penalite_journaliere": "100"} ✓
```

TEST 7.3 setValue() — modifier la durée à 21 jours

```
// setValue() fait un updateOrCreate : met à jour si existe, crée sinon
App\Models\Setting::setValue('duree_emprunt', '21');
App\Models\Setting::getValue('duree_emprunt');
=> "21" ✓
```

TEST 7.4 Nouvel emprunt utilise la nouvelle durée de 21 jours

```
// Le controller relit le setting à chaque emprunt → changement immédiat
$d = (int) App\Models\Setting::getValue('duree_emprunt', 14);
$livreDispo = App\Models\Book::available()->first();
$empTest = App\Models\Loan::create([
    'book_id'     => $livreDispo->id,
```

```
    'user_id'     => $admin->id,
    'borrowed_at' => Carbon\Carbon::now(),
    'due_at'      => Carbon\Carbon::now()->addDays($d),
]);

echo Carbon\Carbon::now()->diffInDays($empTest->due_at) . " jours";
=> 21 jours ✓
```

TEST 7.5 Modifier la pénalité à 200 FCFA/j — recalcul immédiat

```
// calculatePenalty() relit le setting à chaque appel → recalcul immédiat
App\Models\Setting::setValue('penalite_journaliere', '200');

// Crée un emprunt en retard de 3 jours → 3 × 200 = 600 FCFA
$empRetard2 = App\Models\Loan::create([
    'book_id'     => App\Models\Book::available()->first()->id,
    'user_id'     => $biblio->id,
    'borrowed_at' => Carbon\Carbon::now()->subDays(10),
    'due_at'      => Carbon\Carbon::now()->subDays(3),
]);
$empRetard2->calculatePenalty();
=> 600.0  ← 3 jours × 200 FCFA ✓
```

TEST 7.6 Remettre les valeurs initiales (IMPORTANT avant la démo)

```
// TOUJOURS remettre les valeurs initiales avant une démo ou soutenance
App\Models\Setting::setValue('duree_emprunt', '14');
App\Models\Setting::setValue('penalite_journaliere', '100');

// Vérifie la remise à zéro
App\Models\Setting::getValue('duree_emprunt');
=> "14" ✓
App\Models\Setting::getValue('penalite_journaliere');
=> "100" ✓
```

> ▶ **Quitte Tinker — tous les tests sont terminés**

```
exit
```

📊 Résumé final — Bilan de la base après tous les tests

> **Commande de bilan — à lancer dans un nouveau Tinker**

> php artisan tinker → copie le bloc ci-dessous

```
php artisan tinker
```

TEST BILAN Vue d'ensemble complète de la base

```
// Résumé de l'état de la base après tous les scénarios
echo "=== BILAN BASE DE DONNÉES ===\n";
echo "Users          : " . App\Models\User::count() . "\n";
echo "Livres         : " . App\Models\Book::count() . "\n";
echo "  Disponibles  : " . App\Models\Book::available()->count() . "\n";
echo "  Épuisés      : " . App\Models\Book::where('available_copies',0)->count() .
"\n";
echo "Emprunts total : " . App\Models\Loan::count() . "\n";
echo "  En cours     : " . App\Models\Loan::whereNull('returned_at')->count() .
"\n";
echo "  En retard    : " . App\Models\Loan::overdue()->count() . "\n";
echo "  Rendus       : " . App\Models\Loan::whereNotNull('returned_at')->count() .
"\n";
echo "Réservations   : " . App\Models\Reservation::count() . "\n";
echo "  En attente   : " . App\Models\Reservation::enAttente()->count() . "\n";
echo "  Notifiées    : " . App\Models\Reservation::where('status','notifie')-
>count() . "\n";
echo "  Annulées     : " . App\Models\Reservation::where('status','annulee')-
>count() . "\n";
echo "Settings       : " . App\Models\Setting::count() . "\n";
echo "==============================\n";

=> === BILAN BASE DE DONNÉES ===
=> Users          : 3
=> Livres         : 8
=>   Disponibles  : X
=>   Épuisés      : X
=> Emprunts total : X  (seeders + tes tests)
=>   En cours     : X
=>   En retard    : X
=>   Rendus       : X
=> Réservations   : X
=>   En attente   : X
=>   Notifiées    : 1
=>   Annulées     : 1
=> Settings       : 2
=> ==============================
```

✅ Checklist de validation — 28 tests

☐	Réf.	Ce qui est validé
☐	T1.1	3 comptes avec les bons rôles en base

☐	Réf.	Ce qui est validé
☐	T1.2	isAdmin/Biblio/Lecteur() → true pour le bon rôle
☐	T1.3	Vérifications croisées → false pour les mauvais rôles
☐	T1.4	BookPolicy::create() → admin+biblio=true, lecteur=false
☐	T1.5	BookPolicy::delete() → admin=true, biblio+lecteur=false
☐	T1.6	LoanPolicy::view() → lecteur voit seulement ses emprunts
☐	T1.7	LoanPolicy::create() → biblio=true, lecteur=false
☐	T2.1	8 livres créés par BookSeeder
☐	T2.2	search() par titre → résultat correct
☐	T2.3	search() par auteur → résultat correct
☐	T2.4	search() par catégorie → résultat correct
☐	T2.5	search() terme inexistant → collection vide sans erreur
☐	T2.6	available() → exclut les livres épuisés
☐	T2.7	Filtre catégorie direct → comptage correct
☐	T2.8	Relation book → loans (hasMany) fonctionne
☐	T2.9	Relation book → reservations (hasMany) fonctionne
☐	T2.10	Données Clean Code : total=1, dispo=0
☐	T3.3	due_at = today + 14 jours (Carbon::addDays)
☐	T3.4	Observer::created() décrémente available_copies automatiquement
☐	T3.5	Livre épuisé → blocage détecté par available_copies <= 0
☐	T3.6	Relations inverses loan→book et loan→user fonctionnent
☐	T4.2	Loan::overdue() détecte les emprunts en retard
☐	T4.3	calculatePenalty() → 6 × 100 = 600 FCFA
☐	T4.4	calculatePenalty() = 0 si dans les délais

☐	Réf.	Ce qui est validé
☐	T4.5	calculatePenalty() = 0 si livre déjà rendu
☐	T5.2	returned_at + penalty_amount enregistrés en base
☐	T5.3	Observer::updated() incrémente available_copies automatiquement
☐	T6.1	Scope enAttente() filtre correctement
☐	T6.3	File ordonnée par reserved_at (FIFO)
☐	T6.4	Doublon réservation détecté et bloqué
☐	T6.5	1er en file → statut "notifie", 2e reste "en_attente"
☐	T6.6	Annulation réservation → statut "annulee"
☐	T7.1	Setting::getValue() lit duree_emprunt et penalite_journaliere
☐	T7.2	2 paramètres en base (Setting::count() = 2)
☐	T7.3	Setting::setValue() met à jour la valeur en base
☐	T7.4	Nouvelle durée 21j → due_at calculé à 21 jours
☐	T7.5	Pénalité 200 FCFA/j → calcul = 3×200 = 600 FCFA
☐	T7.6	Valeurs remises à 14j / 100 FCFA après les tests

🎓 Backend validé — Prêt pour la démo et la soutenance !